Bootloader & Instruction-Memory Verification Report

RISCFORGE-1TOPS

Cold-Boot, Warm-Boot and Reset-Sequence Coverage | SPEC-BOOTVERIF-v1.0

Introduction

The RISCFORGE-1TOPS SoC boots by fetching a firmware image from an external SPI NOR flash device and copying it into on-chip Instruction SRAM before the RISC-V core is released from reset. This document records the verification of that end-to-end boot path.

Two independent Xcelium testbenches were developed and closed with a 100% pass rate. The first (`soc_imem_boot_tb.sv`) uses an in-house lightweight SPI-NOR behavioral model for fast iteration. The second (`soc_imem_boot_vendor_tb.sv`) substitutes the real Infineon `S25FS512S` vendor model, exercising the boot path against production flash timing including the mandatory $300\,\mu$s power-up wait.

The verification effort was scoped around three concrete objectives:

1. Prove that the **Bootloader FSM** traverses its cold-boot and warm-boot state sequences correctly under both `hard_rst_ni` and `soft_rst_ni` stimulus.
2. Prove that instructions are **correctly delivered** from the SPI flash slave through the SPI master, bootloader, AXI-Lite crossbar, and SRAM interface into the instruction memory module.
3. Prove that the **reset-output sequence** (`core_rst_ni_o`, `periph_rst_ni_o`, `spi_rst_ni_o`) is asserted and released in the correct order for both boot modes, including the post-load clean-slate reset pulse.

IPs Under Test

The complete `riscforge_soc_subsystem` was instantiated in both testbenches. Every IP on the boot-critical path was actively driven and observed. The remaining SoC peripherals were compiled and elaborated (so any structural or elaboration error would surface) but were tied off at their pin boundaries because they are not part of the boot flow.

Actively Exercised (Boot-Path IPs)

IP Module	Role in the Boot Path
`bootloader`	Boot-control FSM (cold and warm branches), reset controller driving `core_rst_ni_o`, `periph_rst_ni_o`, `spi_rst_ni_o`; APB mux (boot-mode drive vs. runtime passthrough); byte-swap on RX FIFO capture; AXI-Lite master for SRAM writes.

IP Module	Role in the Boot Path
apb_spi_master	Full SPI transaction engine. Internally comprises spi_master_apb_if (register file), spi_master_controller (IDLE→CMD→ADDR→DATA_RX FSM), spi_master_clkgen (divider, rise/fall edges), spi_master_tx (MSB-first shift on sdo0), spi_master_rx (MSB-first sample on sdi1), and spi_master_fifo (RX buffer with APB backpressure).
axi_lite_xbar	Routes bootloader master (SLV[1]) to the ISRAM target (MST[0]). Internal axi_lite_demux, axi_lite_mux, rr_arb_tree, addr_decode, spill_register and fall_through_register sub-blocks are all traversed on every AXI write.
axi_lite_sram_if	Converts AXI-Lite write bursts into the req/gnt SRAM protocol. Used both for the 2,048-word SRAM clear pass and for the 8 firmware writes.
sram_mux	Arbitrates the core fetch port (port 0, idle from stub) and the AXI write port (port 1, active from bootloader).
imem_sram_wrapper	Adapts the sram_sp-style interface into the native interface of Instruction_Memory_new, reconstructing the 30-bit absolute word address.
Instruction_Memory_new	The actual instruction RAM (from instr_module). Byte-masked write path and clock-synchronous read path both verified.
s25fs512s	Infineon vendor behavioral model of the SPI NOR flash (used in the vendor testbench). Includes the full tdevice_PU $= 300\,\mu$s power-up gate, standard Read (0x03) command decode, and all setup/hold/CS timing checks.

Compiled but Tied Off (Not on the Boot Path)

- apb_i2c + i2c_master_bit_ctrl + i2c_master_byte_ctrl
- apb_timer + timer
- apb_uart_wrap + apb_uart and the full obi_uart_* chain
- gpio_apb_wrap + gpio + gpio_reg_top
- clic_apb + clic and every clic*_reg_top block
- axi_lite_to_apb, apb_to_obi
- riscv_core_stub (drives all outputs to zero; its reset input is observed but no traffic is generated on its fetch, data, or AXI ports)

End-to-End Boot Data Flow

The boot transfer traverses two logically independent legs: an APB-based *read leg* that pulls a 32-bit word out of the SPI flash, and an AXI-based *write leg* that deposits the byte-swapped word into the target instruction SRAM. The bootloader FSM sits at the pivot between the two legs.

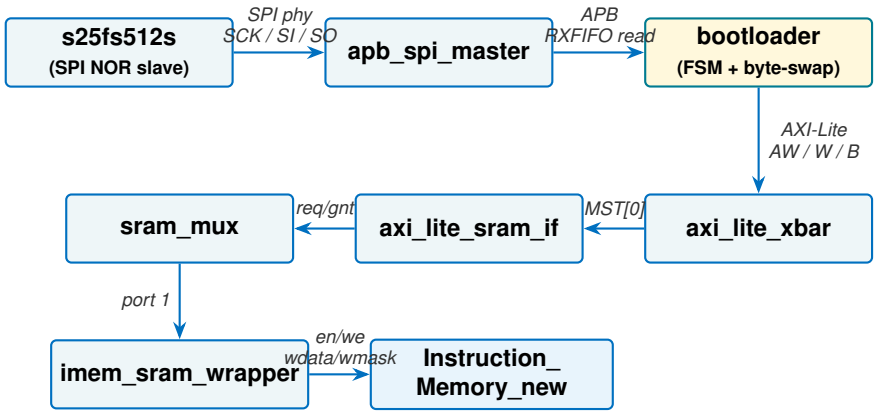


Figure 1: End-to-end boot datapath. The top row shows the SPI read leg, the bottom row shows the AXI write leg. The bootloader byte-swaps the RX FIFO word to convert flash's big-endian byte order into little-endian RISC-V instruction words before it hits SRAM.

Bootloader FSM Branches

Two branches are selected by the `warm_boot_q` latch: the cold-boot branch executes the full SPI fetch; the warm-boot branch takes the short path and re-issues only the clean-slate reset pulse, preserving SRAM contents.

Cold Boot (hard_rst_ni)	Warm Boot (soft_rst_ni)
`S_RESET`	`S_RESET`
`S_CLEAR_SRAM` (2,048 AXI writes)	—
`S_FLASH_WAIT` ($300\,\mu$s tPU)	—
`S_INIT_CLKDIV, SPICMD, SPILEN, SPIDUM`	—
`S_CFG_SPIADR` $\rightarrow$ `S_TRIGGER` $\rightarrow$ `S_READ_FIFO` $\rightarrow$ `S_WRITE_SRAM` $\rightarrow$ `S_CHECK_DONE` *(loop, once per word)*	—
`S_CLEAN_ASSERT`	`S_CLEAN_ASSERT`
`S_CLEAN_HOLD`	`S_CLEAN_HOLD`
`S_CLEAN_RELEASE`	`S_CLEAN_RELEASE`
`S_DONE`	`S_DONE`

Cold-Boot State Descriptions

Cold boot fires on the falling edge of `hard_rst_ni` and executes the full firmware fetch sequence.

- `S_RESET` — Entry state. The FSM inspects the `warm_boot_q` flag (set only by a prior `soft_rst_ni` event). On cold boot the flag is 0, so the FSM proceeds to `S_CLEAR_SRAM`. All three reset outputs are held LOW.
- `S_CLEAR_SRAM` — Issues $2{,}048$ AXI-Lite writes of `0x00000000` to cover the full ISRAM plus DSRAM address range. Guarantees a deterministic starting state (no X-propagation from uninitialised RAM cells) before the firmware fetch begins. `spi_rst_ni_o` is released HIGH on entry to this state so that the SPI IP has time to come out of reset well before it is accessed.
- `S_FLASH_WAIT` — Idle countdown that satisfies the external flash device's power-on recovery time. The BFM testbench collapses this to 10 cycles; the vendor testbench uses the full 30,000 cycles ($300\,\mu$s at 100 MHz) required by the `S25FS512S` `tdevice_PU` parameter.
- `S_INIT_CLKDIV` — One-shot APB write that programs the SPI master's clock divider register, setting the SCK frequency for all subsequent read transactions.
- `S_INIT_SPICMD` — One-shot APB write that loads the SPI master's command register with the Standard Read opcode (`0x03`), MSB-aligned so the TX shifter emits it first.

- S_INIT_SPILEN — One-shot APB write that programs the transaction geometry: 8 command bits, 24 address bits, and 32 data bits per read.
- S_INIT_SPIDUM — One-shot APB write that sets the dummy-cycle count to zero (Standard Read requires none).
- S_CFG_SPIADR — Per-word APB write that loads the 24-bit flash byte address for the next read into the SPI master's address register. Address increments by 4 each loop iteration.
- S_TRIGGER — Per-word APB write to the SPI master's status/control register that toggles the chip-select active and starts the serial clock, kicking off the shift operation.
- S_READ_FIFO — The FSM issues an APB read of the SPI master's RX FIFO. The APB slave holds PREADY $= 0$ while the FIFO is empty, so the bootloader naturally stalls in this state until the serial shift completes and a full 32-bit word arrives. On the same cycle PREADY goes HIGH, the FSM latches the byte-swapped word into rxfifo_data_q.
- S_WRITE_SRAM — Drives the captured word onto the AXI-Lite AW/W channels targeting the current dest_addr_q. Waits for b_valid from the crossbar to confirm the SRAM write landed.
- S_CHECK_DONE — Increments the flash address, destination address, and word counter. If word_cnt_q + 1 < TOTAL_WORDS the FSM loops back to S_CFG_SPIADR for the next word; otherwise it proceeds to the clean-slate reset sequence.
- S_CLEAN_ASSERT — Drives all three reset outputs LOW simultaneously, wiping any transient state that accumulated during the SPI fetch (e.g. leftover SPI master register contents) before the core is released.
- S_CLEAN_HOLD — Holds the reset pulse LOW for RESET_HOLD_CYCLES cycles so downstream flip-flops see enough setup time to actually reset.
- S_CLEAN_RELEASE — Releases all three reset outputs HIGH on the same clock edge, so the core, peripherals and SPI master come out of reset synchronously.
- S_DONE — Terminal state. Asserts boot_done_o = 1 and switches the APB mux from boot-mode to runtime passthrough, letting the core drive the SPI master directly. The FSM self-loops here until the next reset event.

Warm-Boot State Descriptions

Warm boot fires on the falling edge of soft_rst_ni (with hard_rst_ni held HIGH). It re-issues the clean-slate reset pulse to give the core and peripherals a fresh restart while *preserving* the previously loaded SRAM image, so the CPU restarts against the same firmware without a costly re-fetch.

- S_RESET — The asynchronous reset block latches warm_boot_q = 1. The next-state logic sees the flag and jumps directly to S_CLEAN_ASSERT, skipping the SRAM clear, the flash wait, the SPI programming writes, and the entire per-word fetch loop. This is the essential difference from cold boot.
- S_CLEAN_ASSERT — Same behaviour as in cold boot: all three reset outputs driven LOW simultaneously. SRAM contents are unaffected because these resets do not clear the RAM array (only the control-path flip-flops in the SPI master, core, and peripherals).
- S_CLEAN_HOLD — Holds the reset pulse LOW for the same RESET_HOLD_CYCLES duration used in cold boot.
- S_CLEAN_RELEASE — Releases all three resets HIGH synchronously, restarting the core against the retained SRAM image.
- S_DONE — Reasserts boot_done_o and re-enables APB passthrough. In the vendor testbench this state is reached six cycles after soft_rst_ni is released — three orders of magnitude faster than cold boot, which is precisely the benefit of retaining SRAM across warm reset.

Note

The warm branch touches **no** SPI signals. The verification testbench proves this by poisoning the flash with `0xDEADBEEF` before pulsing `soft_rst_ni` and confirming afterwards that (a) no falling edge occurred on `spi_csn0_o`, and (b) the ISRAM still contains the original golden image rather than the poison pattern.

Test Methodology

Golden Firmware Image

Both testbenches use the same 8-word golden image, chosen to be architecturally meaningful (all bytes distinct, all four byte lanes non-zero) so that any endianness or byte-lane error is immediately visible in the mismatch report.

Listing 1: Golden RV32I instruction image

```
golden_instr[0] = 32'h00100093;  // ADDI x1, x0, 1
golden_instr[1] = 32'h00200113;  // ADDI x2, x0, 2
golden_instr[2] = 32'h00300193;  // ADDI x3, x0, 3
golden_instr[3] = 32'h00400213;  // ADDI x4, x0, 4
golden_instr[4] = 32'h00500293;  // ADDI x5, x0, 5
golden_instr[5] = 32'h00600313;  // ADDI x6, x0, 6
golden_instr[6] = 32'h00700393;  // ADDI x7, x0, 7
golden_instr[7] = 32'h00800413;  // ADDI x8, x0, 8
```

Flash Preload and Endianness Contract

The image is preloaded into the flash slave in **little-endian-in-flash** byte order: byte address 0 holds the LSB, byte 3 holds the MSB. The SPI master shifts each byte MSB-first, so the RX FIFO word arrives byte-reversed relative to the RISC-V representation. The bootloader byte-swap (`{prdata[7:0], prdata[15:8], prdata[23:16], prdata[31:24]}`) then recovers the correct 32-bit instruction before the AXI write.

- In the BFM testbench the preload is done through an internal `write_word(word_addr, data)` task on the BFM's byte array.
- In the vendor testbench the preload is done hierarchically, after the vendor model's `InitMemory` initial block has run (a 1 ns settle is inserted), by calling `u_flash.memory_features_i0.write_mem_w(byte_addr, byte_data)`.
  The preload completes well before the `S_FLASH_WAIT` timer expires, so the vendor model has valid data by the time the bootloader issues its first `0x03` read.

Reset Observation Infrastructure

The three boot-related reset outputs are internal wires (`dut.core_rst_n`, `dut.periph_rst_n`, `dut.spi_rst_n`) rather than SoC top-level pins. They are exposed to the testbench via hierarchical reference and monitored by cycle-accurate first-transition detectors so that the relative ordering of releases can be scored after boot completes. In parallel, a `negedge spi_csn0_o` monitor records any SPI chip-select assertion during either boot phase, enabling the "no SPI activity during warm boot" assertion.

Test 1 — Cold Boot Procedure

1. Preload the flash slave with the golden image.
2. Drive `hard_rst_ni` LOW for 10 cycles.

3. Assert the three pre-release reset checks.
4. Enable cold-boot reset scoring; release `hard_rst_ni`.
5. Wait for `boot_done_o` within the `BOOT_TIMEOUT_CYCLES` budget.
6. Read `Instruction_Memory_new.mem_array[0..7]` via hierarchical reference and compare to the golden image.
7. Read `mem_array[8..11]` and confirm zeros (SRAM-clear pass succeeded; no over-write past `NUM_INSTR_WORDS`).
8. Score the reset-sequence assertions listed in Section 5.

Test 2 — Warm Boot Procedure

1. Overwrite the entire flash slave contents with the `0xDEADBEEF` poison pattern. Any accidental SPI refetch during warm boot would corrupt SRAM and be visible against the pre-warm snapshot.
2. Snapshot `mem_array[0..7]` into a testbench-local array `pre_warm[]`.
3. Enable warm-boot SPI scoring; pulse `soft_rst_ni` LOW for 5 cycles (`hard_rst_ni` remains HIGH); release.
4. Wait for `boot_done_o` to reassert; add a 10-cycle settling window before sampling reset wires and SRAM.
5. Verify `warm_boot_q` latched, FSM parked in `S_DONE`, all three resets HIGH, and zero SPI CS0 activity was observed during the warm phase.
6. Compare `mem_array[0..7]` against *both* the pre-warm snapshot *and* the golden image (must match both to prove retention).

Note

The DEADBEEF poison pattern is a deliberate diagnostic. If a warm boot regression ever accidentally re-enters the SPI fetch loop, the resulting SRAM contents will contain the poison, and the retention assertion fails with an unambiguous mismatch message rather than a silent pass on stale data.

Verified Behaviours

Table 2 enumerates every behaviour asserted by the testbench, grouped by category. All 36 checks pass in both the BFM and vendor-flash runs.

Category	Behaviour Verified	Checks
Reset (cold, pre-release)	`core_rst_n`, `periph_rst_n`, `spi_rst_n` all LOW while `hard_rst_ni` held.	3
FSM traversal (cold)	Sequential visits to `S_CLEAR_SRAM`, `S_FLASH_WAIT`, the four `S_INIT_*` config writes, and eight per-word fetch/write iterations terminating in `S_DONE`. Observed via `state_q` monitor.	(implicit)
SPI protocol	Standard-read (`0x03`) opcode accepted by the vendor model; 24-bit address correctly transmitted; 32-bit data correctly received on IO1 (`sdi1`). No vendor timing check violated.	(implicit)
SRAM contents (cold)	8 golden instruction words match at `mem_array[0..7]`.	8
SRAM clear	`mem_array[8..11]` $= 0$; loader did not write past `NUM_INSTR_WORDS`.	4

Category	Behaviour Verified	Checks
Reset sequence (cold)	`spi_rst_n` released early (before core/periph, so the SPI IP is live during `S_FLASH_WAIT`); `core_rst_n` and `periph_rst_n` released on the same cycle (single clean-slate pulse); all three HIGH at `S_DONE`; FSM parked in `S_DONE`; real SPI CS0 activity observed during boot.	8
FSM traversal (warm)	`soft_rst_ni` pulse takes the short path `S_RESET` → `S_CLEAN_ASSERT` → `S_CLEAN_HOLD` → `S_CLE` → `S_DONE` in 6 cycles.	(implicit)
Warm-boot control	`boot_done_o` reasserted; `warm_boot_q` latched; FSM parked in `S_DONE`; all three resets HIGH; **zero** SPI CS0 activity (SPI fetch skipped as required).	5
SRAM retention (warm)	8 words in ISRAM unchanged after warm reset — match both pre-warm snapshot and golden image, despite the flash slave being poisoned with `0xDEADBEEF`.	8
	Total scored checks	**36**

Table 2: Verified behaviours and check counts.

Results

Both testbenches close at $36/36$ checks passed. Table 3 compares the two configurations.

Table 3: Testbench closure summary.

Metric	BFM Testbench	Vendor Testbench
Flash model	In-house lightweight BFM	Infineon `S25FS512S`
`FLASH_POWERUP_CYCLES`	10	30,000 (matches tPU)
Cold-boot completion (cycle)	15,442	45,432
Warm-boot completion (cycles)	6	6
Cold-boot ISRAM matches	8 / 8	8 / 8
Cold-boot cleared spot-check	4 / 4	4 / 4
Cold reset-sequence asserts	8 / 8	8 / 8
Warm-boot control asserts	5 / 5	5 / 5
Warm-boot SRAM retention	8 / 8	8 / 8
Pre-release reset asserts	3 / 3	3 / 3
Total	**36 / 36**	**36 / 36**

Note

The vendor run's cold-boot latency is ∼30,000 cycles longer than the BFM run. This delta corresponds *exactly* to the `S25FS512S` model's `tdevice_PU` $= 300\,\mu$s power-up timer, gated by the bootloader's `S_FLASH_WAIT` counter. It confirms that the bootloader's `FLASH_POWERUP_CYCLES` parameter is correctly sized against the real device specification.

Significance of the Vendor-Flash Pass

The vendor testbench does not merely re-run the same stimulus against a different model. It additionally proves:

- The bootloader's SPI programming (`CLKDIV`, `cmd_len`, `addr_len`, `data_len`, `dummy=0`) is compatible with a real production flash device's command decoder.
- The SPI master's Mode-0 CPOL/CPHA behaviour matches the vendor model's expectations — no clock-polarity or sampling-edge mismatch fires.
- All setup, hold and CS-timing checks inside the vendor model complete silently across every one of the 8 read transactions.
- The $300\,\mu$s power-up wait built into the bootloader parameter is correctly sized for this vendor part.

Coverage Scope

In Scope (Verified in This Report)

- Bootloader FSM cold branch (all 15 states traversed).
- Bootloader FSM warm branch (`S_RESET` shortcut, clean-slate pulse).
- SPI read datapath from flash slave through `apb_spi_master` to bootloader RX capture.
- AXI-Lite write datapath from bootloader through `axi_lite_xbar`, `axi_lite_sram_if`, `sram_mux`, and `imem_sram_wrapper` into `Instruction_Memory_new`.
- 2,048-word SRAM clear pass.
- RX FIFO byte-swap correctness (proven end-to-end by the golden-image match).
- Reset-output ordering (`spi_rst_n` first, then core/periph together) for both boot modes.
- SRAM retention across warm boot; correct absence of SPI activity on warm boot (poison-pattern check).

Out of Scope (Follow-Up Work)

- DSRAM boot path (`NUM_DATA_WORDS = 0` in this test).
- Firmware sizes at the `ISRAM_NUM_WORDS` boundary.
- Runtime APB passthrough after `S_DONE` (requires a core stub that issues APB traffic).
- AXI error responses (`b.resp` $\neq$ `OKAY`) and APB `pslverr` propagation.
- Quad-SPI mode and dummy-cycle read modes.
- Warm-boot arrival during an in-flight cold boot (`soft_rst_ni` pulse before `S_DONE`).

Important

The out-of-scope items are not defects. They are known coverage gaps in this specific testbench pair. Any of them can be closed by adding targeted stimulus and assertions without modifying the bootloader RTL or the DUT wiring already validated here.

Testbench Artifacts

Both filelists set `-sv -timescale 1ns/1ps -access +rwc` and list every RTL source in dependency order. Invocation from `ips/RTL_SOC/`:

```
xrun -f rtl/soc_imem_boot.f          # BFM testbench
xrun -f rtl/soc_imem_boot_vendor.f   # Vendor-flash testbench
```